

Incidental Design II Incident Declaration and Initial Triage Features

Incidental Design II

Incident Declaration and Initial Triage Features

Design Scope

This section extends the Design I incident declaration feature set by introducing the user interfaces and major implementation logic for four connected use cases: declaring a new incident, assigning incident severity, categorizing the incident type, and assigning the initial responder or response team.

1. UI Design

1.1. Screen 1: Declare New Incident Form

Purpose

Allows an incident commander or system operator to create a new incident by entering the minimum information required to begin response coordination.

Declare New Incident	
Title	Enter incident title
Description	Describe the issue, symptoms, and visible impact.
Declared By	Current authenticated user
Validation	Title required; description required
	[Cancel] [Submit]

Primary inputs: incident title, incident description.

Primary outputs: newly created incident record, confirmation message, and visible incident entry for responders.

Incidental Design II Incident Declaration and Initial Triage Features

1.2. Screen 2: Assign Incident Severity

Assign Severity	
Incident	Incident #2481 Payment API Outage
Current Severity	Unassigned
Select Severity	[Low] [Medium] [Critical]
Validation	Severity selection required
	[Cancel] [Confirm]

Primary inputs: selected incident, severity level.

Primary outputs: updated severity value and timeline entry recording the severity change.

1.3. Screen 3: Categorize Incident Type

Categorize Incident	
Incident	Incident #2481 Payment API Outage
Select Category	[Outage] [Security] [Performance] [Other]
Current Category	Unassigned
Validation	One primary category required
	[Cancel] [Save]

Primary inputs: selected incident, incident category.

Primary outputs: updated incident metadata with a single stored category value.

1.4. Screen 4: Assign Initial Responder or Team

Assign Responder	
Incident	Incident #2481 Payment API Outage
Available Responders	[A. Patel] [J. Kim] [SRE Team] [Security On-Call]
Selected Assignee	None
Validation	Responder or team must exist and be available
	[Cancel] [Assign]

Primary inputs: selected incident, responder or response team.

Primary outputs: persisted incident assignment and immediate notification to the assignee.

Incidental Design II Incident Declaration and Initial Triage Features

2. Algorithm Design expressed with Pseudo-code

2.1. Algorithm 1: Declare New Incident

Input: userId, title, description

Output: newIncident or error

```
1.1 user <- FindUserById(userId);
1.2 if user = empty then
1.3     return error "User not found";
1.4 if user.permission != "declare_incident" then
1.5     return error "User does not have permission";
1.6 if Trim(title) = "" then
1.7     return error "Title is required";
1.8 if Trim(description) = "" then
1.9     return error "Description is required";
1.10 incident.incidentId <- GenerateIncidentId();
1.11 incident.title <- title;
1.12 incident.description <- description;
1.13 incident.state <- "DECLARED";
1.14 incident.declaredAt <- CurrentDateTime();
1.15 SaveIncident(incident);
1.16 LinkIncidentToUser(userId, incident.incidentId);
1.17 NotifyRelevantResponders(incident);
1.18 return incident;
```

2.2. Algorithm 2: Assign Incident Severity

Input: userId, incidentId, severityLevel

Output: updatedIncident or error

```
2.1 user <- FindUserById(userId);
2.2 incident <- FindIncidentById(incidentId);
2.3 if user = empty then
2.4     return error "User not found";
2.5 if incident = empty then
2.6     return error "Incident not found";
2.7 if severityLevel not in {LOW, MEDIUM, CRITICAL} then
2.8     return error "Invalid severity level";
2.9 incident.severity <- severityLevel;
2.10 UpdateIncident(incident);
2.11 AddTimelineEntry(incidentId, userId, "Severity Updated",
severityLevel);
2.12 return incident;
```

2.3. Algorithm 3: Categorize Incident Type

Incidental Design II

Incidental Design II Incident Declaration and Initial Triage Features

Input: userId, incidentId, category

Output: updatedIncident or error

```
3.1 user <- FindUserById(userId);
3.2 incident <- FindIncidentById(incidentId);
3.3 if user = empty then
3.4     return error "User not found";
3.5 if incident = empty then
3.6     return error "Incident not found";
3.7 if category not in {OUTAGE, SECURITY, PERFORMANCE, OTHER}
then
3.8     return error "Invalid incident category";
3.9 incident.category <- category;
3.10 UpdateIncidentMetadata(incident);
3.11 return incident;
```

2.4. Algorithm 4: Assign Initial Responder or Team

Input: userId, incidentId, responderId

Output: updatedIncident or error

```
4.1 user <- FindUserById(userId);
4.2 incident <- FindIncidentById(incidentId);
4.3 responder <- FindResponderById(responderId);
4.4 if user = empty then
4.5     return error "User not found";
4.6 if incident = empty then
4.7     return error "Incident not found";
4.8 if responder = empty then
4.9     return error "Responder not found";
4.10 if responder.status = "Unavailable" then
4.11     return error "Selected responder is unavailable";
4.12 incident.responderId <- responderId;
4.13 UpdateIncident(incident);
4.14 SendAssignmentNotification(responderId, incidentId);
4.15 return incident;
```

Incidental Design II Incident Declaration and Initial Triage Features

3. Traceability Between UI and Algorithms

UI Artifact	Supported Logic
Declare New Incident Form	Runs DeclareNewIncident and shows validation errors before submission.
Assign Severity Screen	Runs AssignIncidentSeverity and records the severity change for downstream visibility.
Categorize Incident Screen	Runs CategorizeIncidentType and stores one primary incident category.
Assign Responder Screen	Runs AssignInitialResponderOrTeam and triggers immediate assignee notification.

4. Implementation Notes

The algorithms intentionally abstract away lower-level persistence details such as SQL statements or ORM calls while preserving the major program logic for this stage of design. Input validation is included for every UI that accepts user-entered data, incident declaration enforces the minimum required fields, severity and category updates use predefined controlled values, and responder assignment confirms availability before the incident owner is set. The design also assumes that notification delivery may occur through the system dashboard, email, or messaging integrations already described in the use cases.